

SegRNN: Segment Recurrent Neural Network for Long-Term Time-Series Forecasting

Shengsheng Lin^{ID}, Weiwei Lin^{ID}, Senior Member, IEEE, Wentai Wu^{ID}, Member, IEEE, Feiyu Zhao^{ID}, Ruichao Mo^{ID}, and Haotong Zhang^{ID}

Abstract—With the proliferation of the Internet of Things (IoT) applications, advanced time-series forecasting techniques have become increasingly critical for managing and responding to complex temporal dynamics. However, traditional recurrent neural network (RNN)-based methods have faced challenges in the long-term time-series forecasting (LTSF) domain when dealing with excessively long look-back windows and forecast horizons. Consequently, the dominance in this domain has shifted toward Transformer, multilayer perceptron (MLP), and convolutional neural network (CNN) approaches. The substantial number of recurrent iterations is the fundamental reason behind the limitations of RNNs in LTSF. To address these issues, we propose two novel strategies to reduce the number of iterations in RNNs for LTSF tasks: segment-wise iterations and parallel multistep forecasting (PMF). RNNs that combine these strategies, called SegRNN, significantly reduce the required recurrent iterations for LTSF, resulting in notable improvements in forecast accuracy and inference speed. Extensive experiments demonstrate that SegRNN not only outperforms state-of-the-art Transformer-based models but also reduces runtime and memory usage by more than 78%, making it highly suitable for resource-constrained IoT scenarios. These achievements provide strong evidence that RNNs continue to excel in LTSF tasks and encourage further exploration of this domain with more RNN-based approaches. The code is available at: <https://github.com/lss-1138/SegRNN>.

Index Terms—Internet of Things (IoT) time-series analysis, recurrent neural network (RNN), time-series forecasting.

I. INTRODUCTION

THE rapid development of the Internet of Things (IoT) has enabled large-scale deployment of distributed sensors and smart devices across domains such as energy systems, intelligent transportation, and industrial automation [1]. These

Received 20 May 2025; revised 28 July 2025; accepted 16 December 2025. Date of publication 23 December 2025; date of current version 23 February 2026. This work was supported in part by Guangdong Provincial Natural Science Foundation Project under Grant 2025A1515010113; in part by the Shandong Provincial Natural Science Foundation Project under Grant ZR2024LZH012; in part by Guangxi Key Research and Development Project under Grant 2024AB02018; and in part by the Major Key Project of PCL, China under Grant PCL2025A08 and Grant PCL2025A11. (Corresponding author: Weiwei Lin.)

Shengsheng Lin, Feiyu Zhao, Ruichao Mo, and Haotong Zhang are with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 510641, China.

Weiwei Lin is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 510641, China, and also with Pengcheng Laboratory, Shenzhen 518055, China (e-mail: linww@scut.edu.cn).

Wentai Wu is with the Department of Computer Science, College of Information Science and Technology, Jinan University, Guangzhou 510632, China.

Digital Object Identifier 10.1109/IJOT.2025.3647705

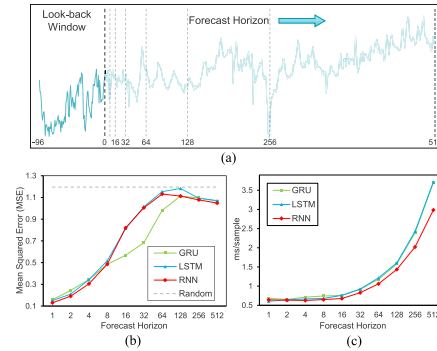


Fig. 1. Challenges faced by the vanilla RNN and its variants in LTSF. Data is obtained from the ETTh1 dataset. (a) LTSF. (b) Forecast error. (c) Inference time.

infrastructures continuously generate vast amounts of multi-variate time-series (MTS) data, making efficient and accurate forecasting essential for intelligent decision-making, anomaly detection, and system optimization [2].

Time-series forecasting, which involves using past observed data to predict future values, is therefore a fundamental capability in IoT-driven applications [3], [4], [5]. In particular, recent focus has shifted toward long-term time-series forecasting (LTSF) [6], [7], [8], [9], where the goal is to provide richer semantic information over extended future horizons. As illustrated in Fig. 1(a), LTSF enables more proactive and informed decisions, especially in IoT contexts that demand foresight, such as predictive maintenance, resource allocation, and risk mitigation.

Recurrent neural networks (RNNs), as a deep learning architecture, have been extensively adopted for conventional time-series forecasting due to their effectiveness in capturing sequential dependencies [10]. However, extending the forecast horizon poses significant challenges.

- 1) Forecasting further into the future introduces higher uncertainty, leading to reduced prediction accuracy.
- 2) Longer horizons require models to process more extensive historical data, which increases modeling complexity and computational overhead.

While RNNs have exhibited remarkable performance in conventional time-series tasks, they have gradually lost prominence in the LTSF domain. Fig. 1(b) and (c) illustrates the limitations of RNNs (either vanilla RNN or its variants: long

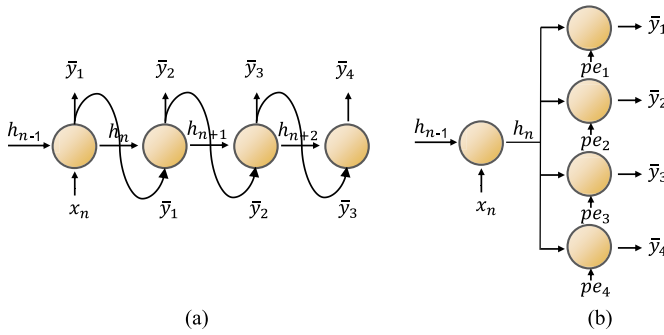


Fig. 2. Comparison of RMF and PMF. The PE pe_i in PMF serves as a replacement for the sequential information of the recurrent structure. (a) RMF (traditional). (b) PMF (ours).

short-term memory (LSTM) [11] and gated recurrent unit (GRU) [12]) in LTSF: 1) the model's forecast error rapidly increases as the forecast horizon expands, especially when the horizon reaches 64, at which point it becomes comparable to random forecasting and 2) the models' inference time rapidly increases with the length of horizon. These observations widely support the belief that RNNs are no longer suitable for LTSF tasks that involve modeling long-term dependencies [6], [13]. Consequently, there is currently no prominent RNN-based solution in the LTSF field.

In contrast, Transformers [14], an advanced neural network architecture designed to model long-term dependencies in sequences, have achieved remarkable success in the LTSF domain, breaking several state-of-the-art records [6], [13], [15]. The undeniable efficacy of Transformers notwithstanding, their intricate design and substantial computational requirements have constrained their accessibility. Recently, there has been significant debate regarding whether the self-attention mechanism in Transformers is suitable for modeling time-series tasks [16], [17]. These debates lead us to contemplate: *Are RNNs, which are conceptually simple and structurally well-suited for modeling time-series data, truly unsuitable for LTSF tasks?*

Answer Might be No: It is well known that RNNs suffer from the vanishing/exploding gradient problem [18], which limits the length of sequences they can effectively model. We can hypothesize that the current RNNs' failure in LTSF stems from excessively long look-back and forecast horizons, which result in prohibitively high recurrent iteration counts. To address this, we propose a straightforward yet powerful strategy: *Minimizing the count of recurrent iterations in RNNs while striving to retain sequential information.* Specifically, we introduce SegRNN, which is designed with two key components.

- 1) The incorporation of segment technology in RNNs, replacing point-wise iterations with segment-wise iterations, significantly reduces the number of recurrent iterations.
- 2) The introduction of the parallel multistep forecasting (PMF) strategy further reduces the number of recurrent iterations. The comparison between PMF and the

traditional recurrent multistep forecasting (RMF) is illustrated in Fig. 2.

Experimental results on popular benchmarks demonstrate that these two key design components significantly improve RNNs' performance in the LTSF domain. Reducing the number of recurrent iterations not only greatly improves the prediction accuracy of RNNs but also significantly enhances their inference speed. In most scenarios, SegRNN outperforms state-of-the-art Transformer-based models, featuring a runtime and memory reduction of over 78%. We provide strong evidence that RNNs still possess powerful capabilities in the LTSF domain. In summary, our contributions are as follows.

- 1) We propose SegRNN, which utilizes a time-series segmentation technique to replace point-wise iterations with segment-wise iterations in LTSF. We further introduce the PMF strategy to enhance the inference speed and forecasting performance of RNNs.
- 2) The proposed SegRNN outperforms current state-of-the-art methods while significantly reducing runtime and memory usage, making it highly suitable for resource-constrained IoT scenarios.
- 3) The success of SegRNN demonstrates substantial improvements over existing RNN-based methods, highlighting the strong potential of RNNs in the LTSF domain and encouraging further exploration in future research.

II. RELATED WORK

Significant efforts have been devoted to advancing the field of time-series forecasting [19], [20]. With the evolution of hardware capabilities, deep learning approaches have gained prominence in uncovering patterns within time-series data [21]. These approaches can be broadly categorized as follows.

A. Transformers

Originally designed for natural language processing tasks [14], Transformers have achieved remarkable success in various domains [22], [23]. The self-attention mechanism in Transformers enables them to capture long-term dependencies in time-series data, leading to a considerable body of work focused on adapting Transformers to LTSF tasks, demonstrating impressive performance [24]. Earlier efforts, such as LogTrans [25], Informer [6], Pyraformer [26], Autoformer [27], and FEDformer [13], aimed at reducing the complexity of Transformers. More recently, PatchTST [15], Crossformer [28], and PETformer [29] leveraged patch-based techniques from computer vision [30], [31], further enhancing the performance of Transformers. The patch technique, which inspired the segment-wise iterations technique in this article, has proved to be influential.

B. Multilayer Perceptrons

Multilayer perceptrons (MLPs) have also found extensive use in time-series forecasting [32], [33]. Recently, DLinear achieved superiority over the state-of-the-art Transformer-based models through a simple linear layer and channel-independent strategy [16]. The success of DLinear has spurred

the development of a plethora of MLPs in LTSF, including MTS-Mixers [17], TSMixer [34], SparseTSF [35], and TiDE [36]. The accomplishments of these MLP-based models have raised questions about the necessity of employing complex and cumbersome Transformers for time-series prediction.

C. Convolutional Neural Networks

Initially applied in image processing for capturing local patterns and extracting meaningful features [37], [38], convolutional neural networks (CNNs) have also shown remarkable performance in the time-series domain [39], [40], [41]. Recently, CNN-based models such as MICN [42], TimesNet [43], and SCINet [44] have demonstrated impressive results in the LTSF field. However, due to the limitation of the local reception field size of the kernel, they often require stacking multiple layers to effectively model long-term dependencies, which, in turn, increases the complexity of modeling.

D. Recurrent Neural Networks

RNNs have long been the primary choice for time-series forecasting tasks due to their ability to handle sequential data. Numerous efforts have been devoted to utilizing RNNs for short-term and probabilistic forecasting, achieving significant advancements [45], [46], [47], [48]. However, in the LTSF domain with excessively long look-back windows and forecast horizons, RNNs have been considered inadequate for effectively capturing long-term dependencies, leading to their gradual abandonment [6], [13]. *The emergence of SegRNN aims to challenge and change this situation by attempting to address these limitations.*

III. PRELIMINARIES

This section introduces the formulation of the LTSF problem, the channel-independent (CI) strategy, and the fundamental RNN and its variants.

A. LTSF Problem Formulation

The LTSF problem deals with predicting the future time series $Y \in \mathbb{R}^{H \times C}$ based on a historical MTS $X \in \mathbb{R}^{L \times C}$. Here, L represents the length of the historical look-back window, C denotes the number of feature dimensions or channels and H signifies the length of the forecast horizon. The goal of the LTSF task is to extend the forecast horizon H to its maximum potential (e.g., up to 720), which poses a considerable challenge.

B. CI Strategy

Intuitively, it may appear optimal to use all historical variables in an MTS to forecast all future variables simultaneously, as it captures the interrelationships between the variables. However, recent studies have shown that the CI strategy surpasses this traditional approach [49]. The CI strategy aims to identify a function $f: X^{(i)} \in \mathbb{R}^L \rightarrow Y^{(i)} \in \mathbb{R}^H$ that maps the univariate historical time-series data to the future time-series values, as opposed to $f: X \in \mathbb{R}^{L \times C} \rightarrow Y \in \mathbb{R}^{H \times C}$, which maps

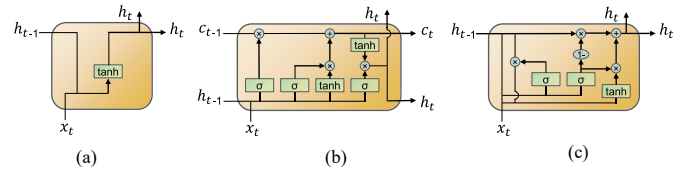


Fig. 3. Comparison of the cell of the RNN and its variants. (a) RNN cell. (b) LSTM cell. (c) GRU cell.

the multivariate historical time-series data to the future time-series values. The current state-of-the-art LTSF models have embraced the CI strategy [15], [16], [50], and we similarly align ourselves with this approach. Furthermore, our model introduces a channel identifier [51] to enhance its predictive capability for multivariate sequences, which will be discussed comprehensively in Section IV-B.

C. RNN Variants

The vanilla RNN faces challenges such as vanishing and exploding gradients, which hinder the model's convergence during training [18]. To address these issues, the LSTM and GRU architectures optimize the structure of their cell units. Fig. 3 illustrates the differences in cell unit structures among these fundamental RNN variants. For more comprehensive information, we strongly recommend referring to the original papers [11], [12].

The proposed SegRNN is not limited to a specific RNN cell structure. Considering the stable performance of GRU in practical scenarios, this article employs GRU as an exemplar to illustrate the architecture and performance of SegRNN. Therefore, for consistency throughout the text, the SegRNN model is assumed to be based on the GRU cell unless explicitly stated otherwise.

IV. MODEL ARCHITECTURE

The recurrent iterative nature of RNNs poses challenges for effective convergence when modeling extensive long sequences. SegRNN aims to reduce the number of recurrent iterations to facilitate its convergence. Technically, SegRNN employs the following strategies.

- 1) In the encoding phase, it replaces the original point-wise iterations with segment-wise iterations, effectively reducing the number of iterations from L to L/w .
- 2) In the decoding phase, it utilizes the PMF strategy to further reduce the number of iterations from H/w to 1.

The substantial reduction in the number of recurrent iterations in RNNs not only leads to a remarkable performance improvement but also results in a significant increase in inference speed. The model architecture of SegRNN is shown in Fig. 4, with detailed pseudocode provided in the Appendix.

A. Encoding

1) *Segment Partition and Projection*: Given a sequence channel $X^{(i)} \in \mathbb{R}^L$, it can be partitioned into segments $X_w^{(i)} \in \mathbb{R}^{n \times w}$, where w represents the window length of each segment and $n = (L/w)$ denotes the number of segments. These

segments, $X_w^{(i)} \in \mathbb{R}^{n \times w}$, are then transformed to $X_d^{(i)} \in \mathbb{R}^{n \times d}$ through a learnable linear projection $W_{\text{prj}} \in \mathbb{R}^{w \times d}$ followed by a ReLU activation [52], where d represents the dimensionality of the hidden state of the GRU.

2) *Recursive Encoding*: Subsequently, the transformed $X_d^{(i)}$ is fed into the GRU for recurrent iterations to capture temporal features. Specifically, for $x_t \in \mathbb{R}^d$ in $X_d^{(i)}$, the entire process within the GRU cell can be formulated as

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (1)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (2)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \times h_{t-1}, x_t]) \quad (3)$$

$$h_t = (1 - z_t) \times h_{t-1} + z_t \times \tilde{h}_t. \quad (4)$$

After n recurrent iterations, the hidden feature h_n obtained from the last step already encapsulates all the temporal features of the original sequence $X^{(i)}$. This hidden feature will be passed to the Decoding part for the subsequent steps of inference and prediction.

B. Decoding

RMF is a straightforward method for accomplishing multistep prediction. In RMF, single-step predictions are made to obtain $\bar{y}_t$. The predicted $\bar{y}_t$ is then used as input for the subsequent prediction of $\bar{y}_{t+1}$, and this process continues until the complete set of prediction results is obtained. By incorporating the segment technique from the Encoding phase with RMF, the number of iterations required for a prediction horizon of H is reduced to H/w .

However, despite the reduction in the number of recurrent iterations, recursive decoding has its limitations. These include: 1) the accumulation of errors resulting from recursive predictions and 2) the sequential nature of recursion, which hampers parallel computation within training examples and restricts improvements in inference speed [14]. To address these limitations, we propose a novel prediction strategy called PMF, as described below.

1) *Positional Embeddings*: During the decoding phase, the sequential order between segments is lost due to the break in the recurrent recursion. To address this, m corresponding positional embeddings (PEs), denoted as $\text{PE}^{(i)} \in \mathbb{R}^{m \times d}$, are generated to identify the positions of the segments. Here, $m = (H/w)$ represents the number of windows obtained by partitioning the prediction horizon into segments. Each PE $\text{pe}^{(i)} \in \mathbb{R}^d$ is constructed by concatenating the learnable relative position (RP) encoding $\text{rp} \in \mathbb{R}^{(d/2)}$ and the learnable channel position (CP) encoding $\text{cp} \in \mathbb{R}^{(d/2)}$.

Fig. 5 illustrates the PEs for the target sequence $Y^{(i)}$. The RP encoding indicates the position of each segment that needs to be predicted within the complete sequence $Y^{(i)}$. Meanwhile, the CP encoding represents the CP $i \in \{1, 2, \dots, C\}$ of the current sequence $Y^{(i)}$ within the multichannel sequence $Y \in \mathbb{R}^{H \times C}$ [51]. The inclusion of the CP encoding partially compensates for the limitations of the CI strategy in capturing relationships between variables, thereby enhancing the model's performance.

2) *Parallel Decoding*: In the decoding phase, the same GRU cell trained in the encoding phase is shared. Specifically, the final state h_n obtained from the encoding phase is duplicated m times and combined with the m PEs from $\text{PE}^{(i)}$. These pairs are then *simultaneously* processed in parallel by the GRU cell. This parallel processing generates m output vectors, denoted as $\bar{Y}_d^{(i)} \in \mathbb{R}^{m \times d}$.

It is important to note that this approach differs from the previous recursive processing in RMF, as the computation of each vector is independent of the previous time step's result. As a result, intrasample parallel computation is achieved, leading to improved inference speed. Additionally, prediction errors do not accumulate with the number of iterations, resulting in enhanced prediction accuracy.

3) *Prediction and Sequence Recovery*: The $\bar{Y}_d^{(i)}$ undergoes a Dropout layer [53], randomly dropping out a certain proportion of values for regularization purposes. Subsequently, it is transformed into $\bar{Y}_w^{(i)} \in \mathbb{R}^{m \times w}$ using a learnable linear prediction layer $W_{\text{prd}} \in \mathbb{R}^{d \times w}$. Furthermore, $\bar{Y}_w^{(i)}$ is reshaped into $\bar{Y}^{(i)} \in \mathbb{R}^H$, representing the final prediction result.

C. Normalization and Evaluation

1) *Instance Normalization*: Time-series data often experience distribution shift issues, and employing simple sample normalization strategies can help alleviate this problem [54]. In this article, we utilize a simple sample normalization strategy [16] that involves subtracting the last value of the sequence from the input before encoding and subsequently adding back the value after decoding, formulated as

$$x_{1:L}^{(i)} = x_{1:L}^{(i)} - x_L^{(i)} \quad (5)$$

$$\bar{y}_{1:L}^{(i)} = \bar{y}_{1:L}^{(i)} + x_L^{(i)}. \quad (6)$$

In addition, for datasets with severe distribution shifts (e.g., Traffic), we apply RevIN [54], a more sophisticated normalization technique widely adopted in recent time-series forecasting studies [15], [36].

2) *Loss Function*: We adopt the classic L2 loss as the training objective, which measures the mean squared error (mse) between the predicted values and the ground truth. Specifically, it is defined as

$$\mathcal{L}(Y, \bar{Y}) = \frac{1}{HC} \sum_{t=1}^H \sum_{i=1}^C \left(\bar{y}_t^{(i)} - y_t^{(i)} \right)^2. \quad (7)$$

V. EXPERIMENTS

In this section, we present the main experimental results on popular LTSF benchmarks. Furthermore, we conduct an ablation study to analyze the impact of the segment-wise iterations and the PMF strategies on the effectiveness of RNNs in LTSF. We also investigate the influence of some important parameters on SegRNN. All experiments in this section are implemented in PyTorch and executed on two NVIDIA T4 GPUs, each equipped with 16 GB of memory.

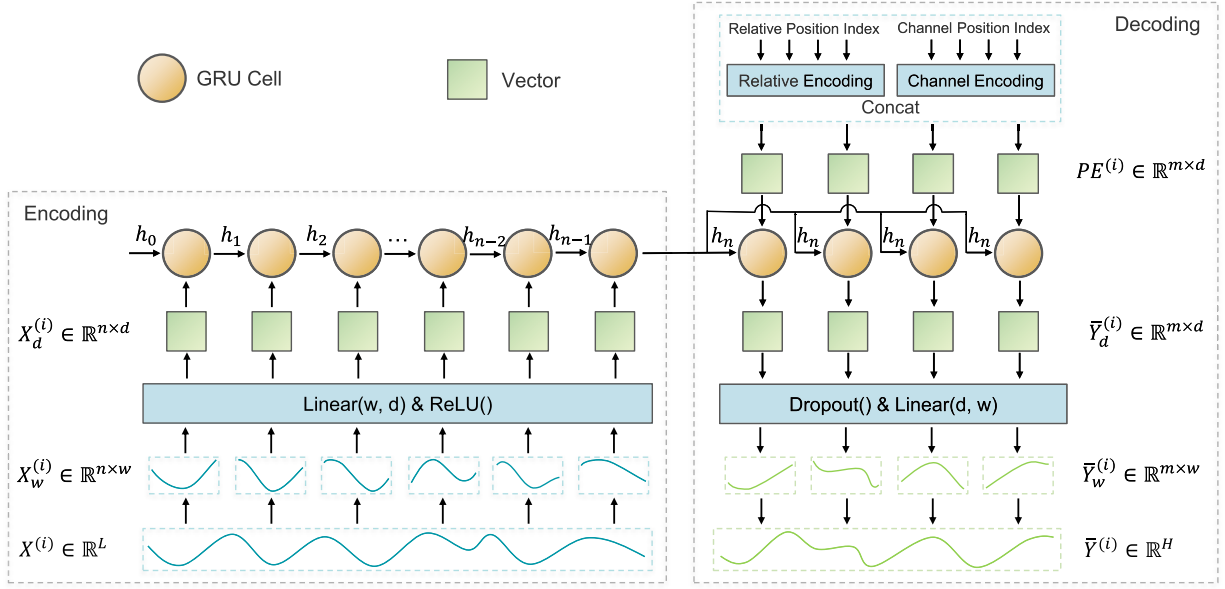
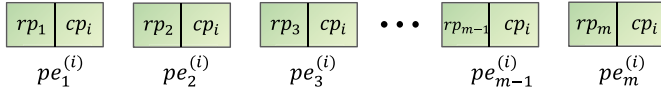


Fig. 4. Model architecture of SegRNN.

Fig. 5. PEs $PE^{(i)}$ for the target sequence $Y^{(i)}$.TABLE I
SUMMARY OF DATASETS FOR EVALUATION

Datasets	ETTh1&2	ETTm1&2	Electricity	Solar-Energy	Traffic	Weather
Channels	7	7	321	137	862	21
Frequency	1 hour	15 mins	1 hour	10 mins	1 hour	10 mins
Timesteps	17,420	69,680	26,304	52,560	17,544	52,696

A. Experimental Setup

The performance evaluation of SegRNN is carried out on eight popular datasets in the LTSF domain, comprising four ETT datasets (ETTh1, ETTh2, ETTm1, and ETTm2), as well as the Electricity, Solar-Energy, Traffic, and Weather datasets [55]. The statistics of these datasets are presented in Table I.

We split all the datasets into training, validation, and test sets in chronological order. Following previous research [15], [16], [27], the ETT datasets are split into proportions of 6:2:2, while the other datasets are divided into proportions of 7:1:2.

1) *Baselines and Metrics*: We select several state-of-the-art and representative models in the LTSF domain as baselines, including iTransformer [55], PatchTST [15], DLinear [16], FEDformer [13], Autoformer [27], and Informer [6]. In addition, we also compare with several RNN-based models, namely DeepAR [56] and GRU [12]. Although DeepAR and GRU were not originally designed for LTSF tasks, we include them as baselines due to the lack of competitive RNN-based solutions in this domain. The proposed SegRNN is introduced to address this gap.

We set the look-back length of all models to $L = 720$, as a longer look-back window better evaluates a model's

ability to capture long-term temporal dependencies and is expected to yield improved forecasting performance. To assess the effectiveness of the LTSF models, we adopt two widely used evaluation metrics: mse and mean absolute error (MAE).

2) *Model Configuration*: We employed the Adam optimizer [57] to train the models for 30 epochs, with a learning rate decay of 0.8 after the initial three epochs. The uniform configuration of SegRNN consists of a look-back of 720, a segment length of 48, a single GRU layer, a hidden size of 512, 30 training epochs, a learning rate decay of 0.8 after the initial three epochs, and early stopping with a patience of 5. The dropout rate, batch size, and learning rate vary based on the scale of the data. The complete set of parameters used for SegRNN across different datasets is available in our open-source repository.¹

B. Main Result

1) *Multivariate Forecasting*: The multivariate LTSF results of SegRNN and other baselines are presented in Table II. Remarkably, SegRNN ranks in the top two across 54 out of 64 metrics under all evaluation settings, including 34 first-place results. This demonstrates its clear superiority over existing baselines, including the current state-of-the-art Transformer-based model, PatchTST.

It is worth noting that among Transformer-based approaches, iTransformer outperforms PatchTST under short look-back windows (e.g., when the look-back length $L = 96$) as reported in its original paper [55]. However, under longer look-back settings used in our experiments ($L = 720$), iTransformer underperforms compared to PatchTST. This can be attributed to the fact that while longer look-back windows provide richer temporal context, they also introduce more noise. As iTransformer relies heavily on channel dependency modeling, it is more prone to overfitting in such settings

¹<https://github.com/lss-1138/SegRNN>

TABLE II

MULTIVARIATE LTSF RESULTS. THE LOOK-BACK LENGTH IS FIXED AT $L = 720$, AND THE FORECAST HORIZON IS SET AS $H \in \{96, 192, 336, 720\}$. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD, AND THE SECOND-BEST RESULTS ARE UNDERLINED. THE COUNT ROW SUMMARIZES THE TOTAL NUMBER OF TIMES EACH METHOD ACHIEVED EITHER THE BEST OR SECOND-BEST PERFORMANCE

Model		SegRNN		iTransformer		PatchTST		DLinear		FEDformer		Autoformer		Informer		DeepAR		GRU	
Metric		MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA
ETTh1	96	0.351	0.392	0.396	0.426	<u>0.375</u>	<u>0.404</u>	0.381	0.407	0.439	0.479	0.594	0.584	1.571	0.964	1.097	0.803	1.135	0.844
	192	0.392	0.414	0.430	0.450	<u>0.418</u>	<u>0.430</u>	0.419	0.430	0.469	0.493	0.606	0.588	1.786	1.045	1.080	0.793	1.123	0.814
	336	0.423	0.433	0.480	0.486	0.462	0.460	0.454	0.456	0.521	0.528	0.618	0.595	1.668	1.003	1.076	0.800	1.080	0.807
	720	0.466	0.472	0.700	0.608	<u>0.495</u>	<u>0.497</u>	0.499	0.512	0.624	0.587	0.742	0.660	1.368	0.900	1.079	0.815	1.089	0.822
ETTh2	96	0.276	0.335	0.311	0.363	<u>0.290</u>	<u>0.354</u>	0.302	0.367	0.398	0.449	0.471	0.487	1.872	1.075	2.674	1.419	1.816	1.112
	192	0.341	0.389	0.392	0.414	<u>0.382</u>	0.419	0.411	0.437	0.412	0.457	0.485	0.492	2.133	1.185	2.699	1.430	2.530	1.369
	336	0.364	0.403	<u>0.415</u>	<u>0.438</u>	0.421	0.448	0.542	0.514	0.440	0.474	0.474	0.488	2.246	1.242	2.505	1.310	3.173	1.356
	720	0.403	<u>0.448</u>	0.425	0.455	<u>0.404</u>	0.444	0.888	0.670	0.469	0.506	0.512	0.513	2.799	1.396	3.132	1.355	3.143	1.348
ETTm1	96	0.293	0.354	0.314	0.366	<u>0.300</u>	<u>0.354</u>	0.307	0.351	0.389	0.439	0.498	0.507	0.947	0.705	1.153	0.857	0.953	0.740
	192	0.328	0.378	0.349	0.389	0.339	<u>0.377</u>	<u>0.337</u>	0.370	0.429	0.458	0.562	0.532	0.980	0.721	1.292	0.827	1.124	0.847
	336	0.357	<u>0.399</u>	0.381	0.411	0.368	0.400	0.371	0.392	0.473	0.472	0.472	0.499	1.044	0.753	1.199	0.866	1.200	0.881
	720	0.410	0.430	0.448	0.449	0.420	0.424	<u>0.416</u>	0.412	0.467	0.473	0.516	0.523	1.199	0.831	1.193	0.869	1.173	0.858
ETTm2	96	<u>0.164</u>	0.250	0.181	0.275	0.168	0.261	0.163	0.256	0.298	0.363	0.389	0.437	1.771	1.099	2.021	1.120	0.818	0.759
	192	<u>0.226</u>	0.294	0.238	0.312	0.231	0.308	0.226	0.308	0.329	0.384	0.422	0.458	2.955	1.431	1.605	1.000	1.048	0.829
	336	<u>0.284</u>	0.339	0.295	0.349	0.279	0.339	0.287	0.350	0.368	0.405	0.481	0.493	3.887	1.683	2.825	1.350	1.862	1.043
	720	<u>0.381</u>	<u>0.402</u>	0.382	0.407	0.364	0.392	0.407	0.430	0.425	0.443	0.532	0.519	4.945	1.901	2.841	1.444	2.858	1.376
Electricity	96	<u>0.131</u>	<u>0.229</u>	0.133	0.229	0.130	0.225	0.136	0.236	0.255	0.362	0.208	0.323	0.349	0.414	0.471	0.495	0.393	0.450
	192	0.151	0.248	0.152	0.249	0.147	0.241	<u>0.150</u>	0.249	0.288	0.387	0.229	0.344	0.317	0.388	0.798	0.709	0.416	0.468
	336	0.167	0.266	0.169	0.266	0.162	0.257	<u>0.166</u>	0.266	0.310	0.405	0.239	0.358	0.334	0.400	0.873	0.755	0.471	0.507
	720	0.201	0.299	0.194	0.288	0.201	<u>0.293</u>	<u>0.200</u>	0.299	0.323	0.410	0.235	0.349	0.311	0.382	0.960	0.801	0.538	0.561
Solar-Energy	96	<u>0.168</u>	0.243	0.190	0.259	0.167	0.226	0.192	0.260	0.326	0.417	0.589	0.579	0.212	<u>0.240</u>	0.219	0.271	0.270	0.292
	192	0.184	<u>0.252</u>	0.204	0.270	<u>0.186</u>	0.245	0.212	0.275	0.336	0.412	0.714	0.605	0.205	0.264	0.326	0.355	0.244	0.283
	336	0.195	0.254	0.218	0.282	<u>0.213</u>	0.257	0.228	0.287	0.320	0.399	0.829	0.694	0.247	<u>0.254</u>	0.772	0.778	0.764	0.774
	720	0.200	<u>0.263</u>	<u>0.218</u>	0.282	0.228	0.266	0.237	0.295	0.437	0.447	0.712	0.584	0.224	0.248	0.762	0.768	0.778	0.783
Traffic	96	0.354	0.251	<u>0.362</u>	0.264	0.370	<u>0.257</u>	0.396	0.288	0.647	0.415	0.590	0.357	0.680	0.386	1.181	0.660	0.897	0.496
	192	0.374	<u>0.263</u>	<u>0.375</u>	0.270	0.382	0.262	0.406	0.290	0.684	0.427	0.604	0.368	0.651	0.361	1.359	0.747	1.465	0.794
	336	0.388	<u>0.272</u>	<u>0.390</u>	0.278	0.395	0.269	0.419	0.296	0.741	0.453	0.624	0.375	0.658	0.362	1.412	0.770	1.474	0.797
	720	<u>0.429</u>	<u>0.292</u>	0.423	0.296	0.434	0.289	0.459	0.319	0.756	0.467	0.634	0.389	0.650	0.349	1.487	0.799	1.495	0.802
Weather	96	0.143	0.198	0.180	0.232	<u>0.149</u>	<u>0.201</u>	0.169	0.228	0.347	0.410	0.444	0.491	0.448	0.439	0.324	0.370	0.264	0.329
	192	0.189	<u>0.247</u>	0.226	0.267	<u>0.191</u>	0.244	0.214	0.272	0.351	0.402	0.574	0.564	0.552	0.504	0.399	0.426	0.326	0.379
	336	0.243	<u>0.291</u>	0.286	0.312	<u>0.245</u>	0.288	0.256	0.306	0.415	0.446	0.551	0.550	0.589	0.512	0.655	0.619	0.655	0.620
	720	0.316	<u>0.345</u>	0.353	0.357	0.315	0.336	<u>0.315</u>	0.354	0.409	0.582	0.597	0.432	0.855	0.687	0.654	0.619	0.654	0.621
Count		54		11		<u>44</u>		16		0		0		3		0		0	

[9]. In contrast, PatchTST adopts the CI strategy that more effectively leverages the extended input sequences without overfitting. Ultimately, SegRNN surpasses both PatchTST and iTransformer across most settings, providing compelling evidence that well-designed RNN-based methods remain highly competitive for LTSF tasks.

Regarding the RNN-based methods GRU and DeepAR, SegRNN achieves 75% and 78% improvements in MSE, respectively. These results highlight that SegRNN not only addresses the limitations of traditional RNNs in LTSF but also reestablishes their competitiveness through architectural innovations. In summary, the success of SegRNN underscores the continued relevance and potential of RNN-based models in the era of Transformer-dominated LTSF research.

2) *Univariate Forecasting*: The univariate LTSF results of SegRNN and other baselines on the full ETT benchmarks are presented in Table III. Since both the input and output consist of only a single channel in univariate forecasting, the CP encoding module in SegRNN is disabled for this setting. Notably, SegRNN ranks in the top two for 31 out of 32 evaluation metrics, including 24 first-place results, once again demonstrating its substantial advantage over other baselines.

Interestingly, in this relatively simpler univariate setting, the lightweight DLinear outperforms the more complex Transformer-based models such as PatchTST and iTransformer. This observation suggests that model performance is

not solely determined by architectural complexity, but rather by how well the model design aligns with the nature of the task. In comparison, SegRNN employs a simple segment-wise iteration mechanism combined with the PMF strategy and is implemented with just a single-layer GRU. This simple and compact design effectively balances forecasting performance and computational efficiency.

C. Ablation Studies

1) *Segment-Wise Iterations Versus Point-Wise Iterations*: Fig. 6 compares segment-wise iterations against point-wise iterations by varying the segment length w . Let L denote the look-back window length and $n = L/w$ denote the number of recurrent iterations. We observe the following.

- (1) When $w = 1$ ($n = L$), segment-wise iterations recover the standard point-wise RNN, and as w increases (i.e., α decreases), the forecast error steadily decreases.
- (2) As $w \rightarrow L$ ($n \rightarrow 1$), the model effectively performs a single-step, feed-forward transformation over the entire sequence, degenerating into an MLP and causing the prediction error to rise again.
- (3) Inference time monotonically decreases as w increases, due to the reduced number of recurrent steps.

Experimentally, larger w always brings faster inference and lower error up to a point; beyond that, the lack of intermediate recurrence harms performance. To find a good tradeoff,

TABLE III

UNIVARIATE LTSF RESULTS. THE LOOK-BACK LENGTH IS FIXED AT $L = 720$, AND THE FORECAST HORIZON IS SET AS $H \in \{96, 192, 336, 720\}$. THE BEST RESULTS ARE HIGHLIGHTED IN BOLD, AND THE SECOND-BEST RESULTS ARE UNDERLINED. THE COUNT ROW SUMMARIZES THE TOTAL NUMBER OF TIMES EACH METHOD ACHIEVED EITHER THE BEST OR SECOND-BEST PERFORMANCE

Model		SegRNN		iTransformer		PatchTST		DLinear		FEDformer		Autoformer		Informer		DeepAR		GRU	
Metric		MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA	MSE	MSA
ETTh1	96	0.053	0.180	0.064	0.198	<u>0.063</u>	<u>0.197</u>	0.079	0.217	0.109	0.259	0.130	0.277	0.284	0.466	0.266	0.444	0.349	0.513
	192	0.067	0.208	0.077	<u>0.218</u>	<u>0.075</u>	<u>0.220</u>	0.095	0.239	0.120	0.271	0.140	0.282	0.371	0.492	0.524	0.652	0.361	0.521
	336	0.079	0.225	<u>0.082</u>	<u>0.229</u>	0.084	0.234	0.118	0.272	0.131	0.284	0.124	0.276	0.362	0.485	1.720	1.271	0.695	0.766
	720	0.078	0.224	<u>0.079</u>	<u>0.225</u>	0.107	0.259	0.206	0.377	0.192	0.360	0.131	0.285	0.341	0.455	1.771	1.296	1.386	1.138
	Count	31		13		6		<u>14</u>		0		0		0		0		0	
ETTh2	96	0.125	0.277	<u>0.142</u>	<u>0.301</u>	0.165	0.320	0.151	0.306	0.183	0.342	0.211	0.362	0.240	0.399	0.271	0.419	0.285	0.430
	192	0.160	0.320	<u>0.161</u>	<u>0.324</u>	0.278	0.420	0.180	0.340	0.221	0.374	0.211	0.362	0.251	0.408	0.438	0.542	0.375	0.498
	336	<u>0.186</u>	<u>0.350</u>	0.155	0.322	0.303	0.425	0.196	0.359	0.265	0.410	0.199	0.363	0.251	0.410	1.406	1.050	1.548	1.111
	720	<u>0.209</u>	<u>0.370</u>	0.163	0.330	0.504	0.556	0.270	0.420	0.364	0.481	0.232	0.396	0.233	0.398	1.419	1.068	1.565	1.132
	Count	31		13		6		<u>14</u>		0		0		0		0		0	
ETTm1	96	0.026	<u>0.122</u>	0.033	0.141	0.028	0.127	<u>0.026</u>	0.122	0.066	0.198	0.101	0.244	0.270	0.438	0.084	0.229	0.070	0.207
	192	0.040	0.154	0.058	0.182	<u>0.041</u>	<u>0.152</u>	0.042	0.152	0.079	0.220	0.139	0.293	0.574	0.674	0.206	0.381	0.220	0.396
	336	<u>0.054</u>	<u>0.179</u>	0.068	0.198	0.056	0.181	0.053	0.174	0.082	0.223	0.107	0.255	0.956	0.875	1.750	1.279	0.196	0.371
	720	0.070	0.202	0.075	0.211	<u>0.073</u>	0.210	0.074	<u>0.206</u>	0.098	0.240	0.131	0.281	1.026	0.921	1.755	1.282	0.320	0.490
	Count	31		13		6		<u>14</u>		0		0		0		0		0	
ETTm2	96	0.063	0.183	0.074	0.210	0.064	0.189	<u>0.064</u>	<u>0.185</u>	0.116	0.266	0.162	0.308	0.210	0.371	0.196	0.358	0.173	0.335
	192	0.089	0.224	0.112	0.261	0.094	0.233	<u>0.093</u>	<u>0.230</u>	0.155	0.308	0.171	0.323	0.264	0.418	0.540	0.610	0.248	0.407
	336	0.117	0.263	0.144	0.298	0.141	0.292	<u>0.125</u>	<u>0.269</u>	0.181	0.335	0.184	0.332	0.262	0.417	0.493	0.576	0.260	0.414
	720	0.156	0.309	0.187	0.343	0.186	0.340	<u>0.176</u>	<u>0.326</u>	0.199	0.346	0.225	0.379	0.309	0.455	0.901	0.809	0.493	0.578
	Count	31		13		6		<u>14</u>		0		0		0		0		0	

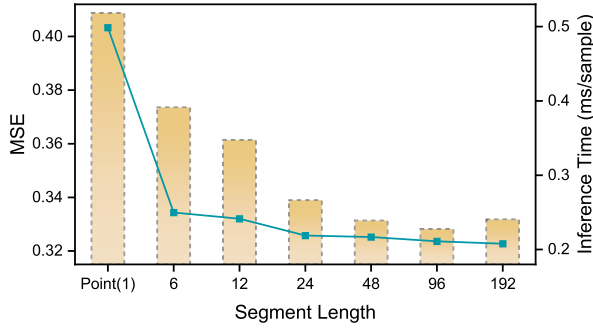


Fig. 6. Forecast error (bar plot) and inference time (line plot) of SegRNN with different segment lengths w on the ETTm1 dataset. Both look-back and horizon are set to 192.

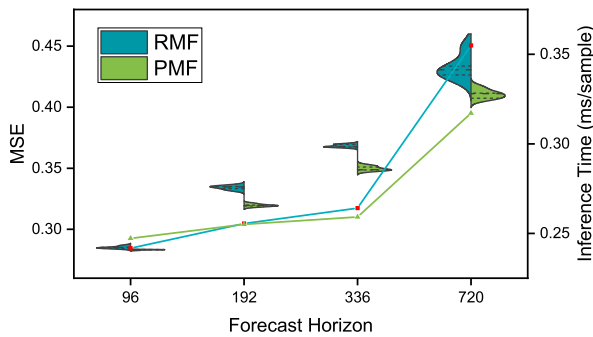


Fig. 7. Forecast error distribution (violin plot) and the inference time (line plot) of SegRNN with different decoding strategies (i.e., RMF and PMF) on the ETTm1 dataset. The look-back is 720.

we examined divisors of our look-back length $L = 720$. Moreover, to ensure alignment with all forecast horizons $H \in \{96, 192, 336, 720\}$, we picked the largest common divisor $w = 48$. This choice yields $n = 720/48 = 15$ iterations, offers almost the minimal MSE, and significantly reduces the runtime cost.

2) *PMF Versus RMF*: Fig. 7 illustrates the performance disparities between RMF and PMF. Concerning the forecast error, PMF significantly outperforms RMF for various forecast horizons, exhibiting a more stable distribution. The advantage of PMF becomes increasingly evident as the forecast horizon increases. This trend highlights that PMF effectively alleviates the issues caused by the excessive recurrent iterations in RMF, which tend to amplify error propagation and degrade performance over longer horizons.

Regarding inference efficiency, an interesting observation emerges: when the forecast horizon $H < 192$, PMF exhibits slightly longer inference times compared to RMF. This is primarily because, at $H = 192$, SegRNN with segment length $w = 48$ only requires four iterations to generate the full output. In this short-horizon setting, the intrasample parallelism of PMF is not fully utilized, while it still incurs extra overhead from data broadcasting and memory duplication. As a result, the marginal gain in parallelism is outweighed by the associated costs.

In contrast, when the forecast horizon increases beyond 192 (i.e., more than four segments are needed), PMF begins to demonstrate clear advantages. The larger number of decoding steps enables better exploitation of intrasample parallelism, which improves hardware utilization (e.g., GPU kernel parallelism and memory access patterns) and reduces overall inference time. This trend becomes more pronounced for longer horizons such as $H = 720$.

In summary, PMF not only improves forecast accuracy but also offers superior inference efficiency in longer-horizon scenarios. These findings reaffirm the effectiveness of PMF in mitigating the iteration bottleneck of RNNs and enabling scalable LTSF modeling.

D. Model Analysis

1) *Impact of Segmentation Strategies*: By default, SegRNN adopts a nonoverlapping segmentation strategy. While this design choice effectively reduces the number of recurrent

iterations, it may raise concerns about potential information loss at temporal breakpoints. For instance, if an extreme value (e.g., a sudden spike) lies at the boundary of a segment, the temporal continuity of that pattern might be weakened. In contrast, overlapping segmentation can potentially preserve temporal correlations more effectively by ensuring that each breakpoint is seen multiple times across segments.

To investigate the impact of the nonoverlapping strategy on capturing abrupt temporal features, we compare the performance of SegRNN using nonoverlapping and overlapping segmentation across several real-world industrial datasets. Among them, the Traffic dataset is particularly prone to extreme values due to peak-hour congestion or unexpected incidents, making it a challenging benchmark. Specifically, we observe an average of 23.8 extreme values per channel in the Traffic dataset (defined as points with a Z-score greater than 6 after normalization). In contrast, the Electricity and Solar-Energy datasets are relatively stable, with only 1.4 and 0 extreme values per channel, respectively.

Table IV reports the performance of SegRNN under both segmentation strategies. Interestingly, the overlapping strategy consistently underperforms the nonoverlapping one across all datasets, regardless of whether the dataset exhibits a high frequency of extreme values (such as Traffic) or demonstrates relatively stable patterns (such as Solar-Energy). We identify two key reasons for this phenomenon.

- 1) *Redundant Historical Information*: The nonoverlapping strategy already incorporates all necessary historical information for accurate prediction. Although overlapping segmentation increases temporal coverage, it primarily introduces redundant information that does not enhance the model's forecasting capability.
- 2) *Increased Recurrent Depth*: More critically, overlapping segmentation significantly increases the number of segments and thus the number of recurrent iterations. This contradicts the core design principle of SegRNN: minimizing recurrent iterations to alleviate the inefficiencies of traditional RNNs. For example, with a look-back length of 720, segment length of 48, and overlap size of 24, the nonoverlapping strategy results in 15 segments, whereas overlapping segmentation yields 29 segments. The deeper recurrent chain introduced by overlapping segmentation weakens temporal information propagation, ultimately leading to degraded performance.

In summary, these results demonstrate that the default nonoverlapping segmentation strategy in SegRNN is sufficiently robust, even in the presence of abrupt temporal variations. Moreover, the findings strongly support the design philosophy of SegRNN: minimizing the number of recurrent iterations in RNNs is a key enabler for reviving their effectiveness in LTSF tasks.

2) *Impact of PEs*: Table V summarizes the impact of different PE strategies on the performance of SegRNN. As shown in Fig. 5, incorporating RP encoding yields the most substantial improvement. Compared to the baseline without any PE ("None"), RP reduces the MSE by an impressive 28.8%. This improvement stems from the nature of the PMF strategy: since the sequential order across forecast steps is no

TABLE IV
COMPARISON OF DIFFERENT SEGMENTATION STRATEGIES
IN THE SEGRNN MODEL

Dataset		Traffic		Electricity		Solar-Energy	
Metric		MSE	MAE	MSE	MAE	MSE	MAE
Non-Overlap	96	0.354	0.251	0.131	0.229	0.168	0.243
	192	0.374	0.263	0.151	0.248	0.184	0.252
	336	0.388	0.272	0.167	0.266	0.195	0.254
	720	0.429	0.292	0.201	0.299	0.200	0.263
	Avg.	0.386	0.270	0.163	0.260	0.187	0.253
Overlap	96	0.364	0.255	0.133	0.230	0.174	0.259
	192	0.384	0.268	0.151	0.248	0.192	0.263
	336	0.393	0.277	0.170	0.268	0.201	0.259
	720	0.436	0.299	0.204	0.301	0.204	0.269
	Avg.	<u>0.394</u>	<u>0.275</u>	<u>0.165</u>	<u>0.262</u>	<u>0.193</u>	<u>0.262</u>
Avg. Change		-2.1%	-1.9%	-1.2%	-0.6%	-3.1%	-3.8%

TABLE V
ABLATION STUDY OF RP ENCODING AND CP ENCODING IN PES. THE
RESULTS ARE THE AVERAGE MSE FOR FORECAST
HORIZONS $H \in \{96, 192, 336, 720\}$

Datasets	RP+CP	RP	CP	None
ETTm1	0.339	<u>0.340</u>	0.683	0.691
ETTm2	0.241	<u>0.245</u>	0.274	0.278
Electricity	0.161	<u>0.162</u>	0.210	0.209
Traffic	<u>0.604</u>	0.596	0.806	0.778
Weather	0.219	<u>0.228</u>	0.243	0.250
Avg.	0.312	<u>0.314</u>	0.443	0.441

longer implicitly preserved through recurrent structures, the model must explicitly learn relative temporal relationships. RP encoding effectively addresses this challenge by reintroducing relative temporal cues, enabling the model to better capture dependencies across forecast steps.

In contrast, CP encoding contributes to performance improvements in most scenarios. This is particularly helpful under the CI strategy, where explicitly encoding the identity of each variable (i.e., channel) aids the model in distinguishing between them [51]. However, on highly multivariate datasets such as Traffic, where the number of variables exceeds 800, introducing CP encoding leads to a moderate performance drop. This suggests that encoding individual CPs at such a scale may introduce noise or increase modeling complexity. Overall, efficiently capturing interchannel relationships remains a challenging problem in LTSF, especially under long look-back settings [9], and we leave further investigation into advanced multichannel modeling strategies for future work.

In summary, the proposed PMF strategy in SegRNN effectively addresses the limitations of traditional RMF. PEs, particularly RP encoding, play a crucial role in enabling this improvement by compensating for the loss of sequential structure. These findings further validate the design choices in SegRNN and open new avenues for enhancing positional representations in time-series forecasting.

3) *Impact of RNN Variants*: The flexibility of SegRNN is further demonstrated through its compatibility with different RNN variants. As shown in Fig. 8, while SegRNN delivers strong performance across various RNN types, it consistently achieves the best results when using the GRU variant.

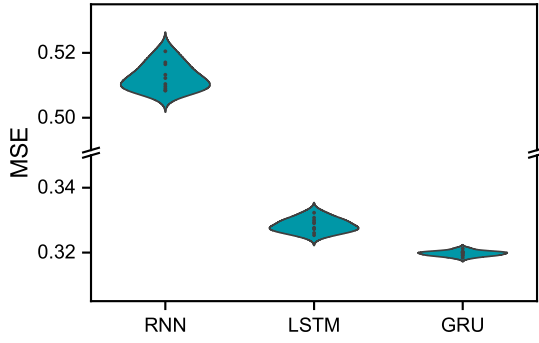


Fig. 8. Forecast error of SegRNN with different RNN variants on the ETTm1 dataset. The look-back is 720, and the horizon is 192.

TABLE VI

COMPARISON OF RUNTIME METRICS BETWEEN SEGRNN AND PATCHTST ON THE ETTM1 DATASET WITH A SINGLE NVIDIA T4 GPU (16 GB). THE LOOK-BACK IS 720, AND THE HORIZON IS 192. SEGRNN EMPLOYS A DEFAULT HIDDEN SIZE OF 512, WHILE PATCHTST UTILIZES A DEFAULT HIDDEN SIZE OF 128

Metric	Datasets	PatchTST	SegRNN	Improve
Training Time (s/epoch)	ETTM1	94.9	29.7	69%
	Weather	273.4	50.3	82%
	Electricity	1.97k	313.8	84%
Parameters (M)	ETTM1	2.62	1.63	38%
	Weather	2.62	1.63	38%
	Electricity	2.62	1.71	35%
Max Memory (MB)	ETTM1	289	77	73%
	Weather	780	124	84%
	Electricity	11.3k	1.23k	89%

The GRU's design, which incorporates gating mechanisms, significantly enhances its ability to model long-term dependencies while maintaining a simpler structure compared to other RNN variants such as the LSTM. This advantage results in more stable and accurate predictions, particularly in the context of time-series forecasting tasks with longer look-backs and forecast horizons.

Consequently, SegRNN adopts GRU as the default backbone, as it strikes an optimal balance between modeling capacity and computational efficiency. However, should more potent RNN variants emerge in the future, SegRNN holds the potential to further bolster its robustness, providing a pathway for continuous performance improvement in LTSF tasks.

4) *SegRNN Versus PatchTST*: To further demonstrate the advantages of SegRNN, we conducted a comparative analysis with PatchTST, a state-of-the-art Transformer-based model designed for time-series forecasting. The results, as shown in Table VI, clearly illustrate the significant efficiency gains provided by SegRNN. On average, SegRNN reduces training time by over 78% and decreases maximum GPU memory consumption by more than 82%, compared to PatchTST.

These efficiency improvements primarily stem from SegRNN's lightweight design, which uses only a single-layer GRU as its core recurrent backbone. Furthermore, the introduction of segment-wise iteration and the PMF strategy substantially reduces the number of recurrent operations, leading to lower computation and memory overhead.

These efficiency improvements are particularly valuable for practical applications, where model training time and computational resources are often limited. Therefore, SegRNN offers a more practical solution for deploying large-scale LTSF models in real-world scenarios, especially in IoT environments characterized by massive data volumes and constrained computational resources.

VI. CONCLUSION

In this article, we introduce SegRNN, an innovative RNN-based model designed for LTSF. SegRNN incorporates two fundamental strategies: 1) the replacement of point-wise iterations with segment-wise iterations and 2) the substitution of RMF with PMF. The segment-wise iteration strategy significantly reduces the number of required recurrent iterations for extracting temporal features, thus addressing the challenge of effectively training RNNs on excessively long sequences. Moreover, the adoption of PMF further mitigates the issue of error accumulation inherent in traditional RMF methods. By adopting these innovative strategies, SegRNN not only outperforms the current state-of-the-art models in terms of prediction accuracy but also yields substantial efficiency improvements, including a remarkable reduction of over 78% in training time and memory usage. As a result, SegRNN offers a scalable and effective forecasting solution tailored to the demands of real-world IoT applications. Finally, these compelling outcomes serve as robust evidence that RNNs remain potent in LTSF tasks, thereby encouraging further exploration and breakthroughs with more RNN methods in the future.

Algorithm 1 Workflow of SegRNN

Require: Sequence length L , prediction horizon H , channel dimension C , segment length w , hidden dimension d , and input $X \in \mathbb{R}^{L \times C}$
Ensure: Forecast output $\hat{Y} \in \mathbb{R}^{H \times C}$

- 1: **# Encoding phase**
- 2: $n \leftarrow \frac{L}{w}$
- 3: $X_w \leftarrow \text{Segment}(X, \text{transpose}(0, 1), w) \triangleright X_w \in \mathbb{R}^{C \times n \times w}$
- 4: $X_d \leftarrow \text{ReLU}(\text{Linear}(X_w)) \triangleright X_d \in \mathbb{R}^{C \times n \times d}$
- 5: $h_n \leftarrow \text{RNN}(X_d, 0) \triangleright h_n \in \mathbb{R}^{C \times 1 \times d}$
- 6: **# Decoding phase**
- 7: $m \leftarrow \frac{H}{w}$
- 8: $PE \leftarrow \text{GeneratePositionalEmbedding}(m, C) \triangleright PE \in \mathbb{R}^{C \times m \times d}$
- 9: $\hat{h}_n \leftarrow h_n.\text{repeat}(1, m, 1) \triangleright \hat{h}_n \in \mathbb{R}^{C \times m \times d}$
- 10: $\hat{Y}_d \leftarrow \text{RNN}(PE, \hat{h}_n) \triangleright \hat{Y}_d \in \mathbb{R}^{C \times m \times d}$
- 11: $\hat{Y}_w \leftarrow \text{Linear}(\text{Dropout}(\hat{Y}_d)) \triangleright \hat{Y}_w \in \mathbb{R}^{C \times m \times w}$
- 12: $\hat{Y} \leftarrow \text{Unsegment}(\hat{Y}_w, w).\text{transpose}(0, 1) \triangleright \hat{Y} \in \mathbb{R}^{H \times C}$

APPENDIX OVERALL PSEUDOCODE

In this section, we present the detailed pseudocode for the SegRNN model, illustrating its implementation. Algorithm 1 outlines the overall workflow, detailing both the encoding and decoding phases. In the encoding phase, we first segment the input sequence and apply a learnable projection to capture its

features. The encoded segments are then processed through an RNN to extract temporal information, resulting in the hidden state h_n . Here, the RNN can be any variant, including standard vanilla RNNs, as well as classical variants like LSTM and GRU (this article defaults to using GRU).

The decoding phase is designed to predict the output by utilizing PEs to account for segment order and channel information. This phase emphasizes parallel processing, allowing multiple predictions to be generated simultaneously, thereby enhancing inference speed and accuracy.

Additionally, Algorithm 2 focuses specifically on the generation of PEs, which are crucial for maintaining the sequential relationships between segments during decoding. First, we generate m RP embeddings and CP embeddings, both of which are learnable parameters that adapt to optimal representations during training. These embeddings are then combined into a positional encoding $PE \in \mathbb{R}^{C \times m \times d}$, which serves as input in the decoding phase.

Algorithm 2 Generate PE

Require: Number of segments m , channel dimension C
Ensure: PEs $PE \in \mathbb{R}^{C \times m \times d}$

- 1: $rp \leftarrow \text{LearnableParameters}(\frac{d}{2}, m)$ ▶Relative position embeddings
- 2: $cp \leftarrow \text{LearnableParameters}(\frac{d}{2}, C)$ ▶CP embeddings
- 3: $PE \leftarrow \text{Tensor}(C, m, d)$
- 4: **for** each channel $c \in \{1, 2, \dots, C\}$ **do**
- 5: **for** each segment $j \in \{1, 2, \dots, m\}$ **do**
- 6: $PE[c, j, : \frac{d}{2}] \leftarrow rp[j]$
- 7: $PE[c, j, \frac{d}{2} : d] \leftarrow cp[c]$
- 8: **end for**
- 9: **end for**
- 10: **return** PE

REFERENCES

- [1] S. Lin, W. Lin, K. Wu, S. Wang, M. Xu, and J. Z. Wang, "Cocv: A compression algorithm for time-series data with continuous constant values in IoT-based monitoring systems," *Internet Things*, vol. 25, Apr.2024, Art. no. 101049.
- [2] Z. Zhang, L. Meng, and Y. Gu, "SageFormer: Series-aware framework for long-term multivariate time-series forecasting," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 18435–18448, May2024.
- [3] S. Aslam, H. Herodotou, S. M. Mohsin, N. Javaid, N. Ashraf, and S. Aslam, "A survey on deep learning methods for power load and renewable energy forecasting in smart microgrids," *Renew. Sustain. Energy Rev.*, vol. 144, Jul.2021, Art. no. 110992.
- [4] Q. Huang et al., "Crossgnn: Confronting noisy multivariate time series via cross interaction refinement," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 46885–46902.
- [5] H. V. Bitencourt, P. D. O. Lucas, O. Orang, P. C. L. Silva, and F. G. Guimarães, "A multistep multivariate fuzzy-based time-series forecasting on Internet of Things data," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 21679–21690, Jun.2025.
- [6] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, May 2021, pp. 11106–11115.
- [7] X. Qiu, X. Wu, Y. Lin, C. Guo, J. Hu, and B. Yang, "DUET: Dual clustering enhanced multivariate time series forecasting," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2025, pp. 1185–1196.
- [8] Q. Huang, Z. Zhou, K. Yang, Z. Yi, X. Wang, and Y. Wang, "Timebase: The power of minimalism in long-term time series forecasting," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 267, PMLR, 2025, pp. 26227–26246.
- [9] S. Lin, H. Chen, H. Wu, C. Qiu, and W. Lin, "Temporal query network for efficient multivariate time series forecasting," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 267, PMLR, 2025, pp. 37797–37814.
- [10] Z. C. Lipton, J. Berkowitz, and C. Elkan, "A critical review of recurrent neural networks for sequence learning," 2015, *arXiv:1506.00019*.
- [11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov.1997.
- [12] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," 2014, *arXiv:1406.1078*.
- [13] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 27268–27286.
- [14] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2025, pp. 5998–6008.
- [15] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. Int. Conf. Learn. Represent.*, 2022.
- [16] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 9, pp. 11121–11128.
- [17] Z. Li, Z. Rao, L. Pan, and Z. Xu, "MTS-mixers: Multivariate time series forecasting via factorized temporal and channel mixing," 2023, *arXiv:2302.04501*.
- [18] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, Mar.1994.
- [19] N. H. An and D. T. Anh, "Comparison of strategies for multi-step-ahead prediction of time series using neural network," in *Proc. Int. Conf. Adv. Comput. Appl. (ACOMP)*, Nov. 2015, pp. 142–149.
- [20] X. Qiu et al., "TFB: Towards comprehensive and fair benchmarking of time series forecasting methods," *Proc. VLDB Endowment*, vol. 17, no. 9, pp. 2363–2377, May2024.
- [21] Z. Han, J. Zhao, H. Leung, K. F. Ma, and W. Wang, "A review of deep learning models for time series prediction," *IEEE Sensors J.*, vol. 21, no. 6, pp. 7833–7848, Mar.2021.
- [22] S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in vision: A survey," *ACM Comput. surveys (CSUR)*, vol. 54, no. 10s, pp. 1–41, 2022.
- [23] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lucic, and C. Schmid, "ViViT: A video vision transformer," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 6816–6826.
- [24] Q. Wen et al., "Transformers in time series: A survey," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2023, pp. 6778–6786.
- [25] S. Li et al., "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 5243–5253.
- [26] S. Liu et al., "Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting," in *Proc. Int. Conf. Learn. Represent.*, 2021.
- [27] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 22419–22430.
- [28] Y. Zhang and J. Yan, "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *Proc. Int. Conf. Learn. Represent.*, 2023.
- [29] S. Lin, W. Lin, W. Wu, S. Wang, and Y. Wang, "PETformer: Long-term time series forecasting via placeholder-enhanced transformer," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 9, no. 2, pp. 1189–1201, Apr.2025.
- [30] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [31] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 15979–15988.
- [32] K. G. Olivares, C. Challu, G. Marcjasz, R. Weron, and A. Dubrawski, "Neural basis expansion analysis with exogenous variables: Forecasting electricity prices with NBEATSx," *Int. J. Forecasting*, vol. 39, no. 2, pp. 884–900, Apr.2023.
- [33] C. Challu, K. G. Olivares, B. N. Oreshkin, F. G. Ramirez, M. M. Canseco, and A. Dubrawski, "NHITS: Neural hierarchical interpolation for time series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 6, pp. 6989–6997.
- [34] V. Ekambaram, A. Jati, N. Nguyen, P. Sinthong, and J. Kalagnanam, "TSMixer: Lightweight MLP-mixer model for multivariate time series forecasting," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2023, pp. 459–469.

- [35] S. Lin, W. Lin, W. Wu, H. Chen, and J. Yang, "SparseTSF: Modeling long-term time series forecasting with 1k parameters," 2024, *arXiv:2405.00946*.
- [36] A. Das, W. Kong, A. Leach, S. Mathur, R. Sen, and R. Yu, "Long-term forecasting with TiDE: Time-series dense encoder," *Trans. Mach. Learn. Res.*, 2023.
- [37] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017.
- [38] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 770–778.
- [39] S. Bai, J. Zico Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," 2018, *arXiv:1803.01271*.
- [40] J.-Y. Franceschi, A. Dieuleveut, and M. Jaggi, "Unsupervised scalable representation learning for multivariate time series," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 4650–4661.
- [41] J. Cheng, K. Huang, and Z. Zheng, "Towards better forecasting by fusing near and distant future visions," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, pp. 3593–3600.
- [42] H. Wang, J. Peng, F. Huang, J. Wang, J. Chen, and Y. Xiao, "Micn: Multi-scale local and global context modeling for long-term series forecasting," in *Proc. Int. Conf. Learn. Represent.*, 2023.
- [43] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-variation modeling for general time series analysis," in *Proc. Int. Conf. Learn. Represent.*, 2022.
- [44] M. Liu et al., "SCINet: Time series modeling and forecasting with sample convolution and interaction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 5816–5828.
- [45] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling long- and short-term temporal patterns with deep neural networks," in *Proc. Int. ACM Conf. Res. Devel. Inf. Retrieval (SIGIR)*, 2018, pp. 95–104.
- [46] S. Bergsma, T. Zeyl, J. R. Anaraki, and L. Guo, "C2FAR: Coarse-to-fine autoregressive networks for precise probabilistic forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 21900–21915.
- [47] R. Wen, K. Torkkola, B. Narayanaswamy, and D. Madeka, "A multi-horizon quantile recurrent forecaster," 2017, *arXiv:1711.11053*.
- [48] Y. Tan, L. Xie, and X. Cheng, "Neural differential recurrent neural network with adaptive time steps," 2023, *arXiv:2306.01674*.
- [49] L. Han, H.-J. Ye, and D.-C. Zhan, "The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 7129–7142, Nov. 2024.
- [50] S. Lin, W. Lin, F. Zhao, and H. Chen, "Benchmarking and revisiting time series forecasting methods in cloud workload prediction," *Cluster Comput.*, vol. 28, no. 1, p. 71, Feb. 2025.
- [51] Z. Shao, Z. Zhang, F. Wang, W. Wei, and Y. Xu, "Spatial-temporal identity: A simple yet effective baseline for multivariate time series forecasting," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2022, pp. 4454–4458.
- [52] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn.*, 2010, pp. 807–814.
- [53] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," 2012, *arXiv:1207.0580*.
- [54] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, "Reversible instance normalization for accurate time-series forecasting against distribution shift," in *Proc. Int. Conf. Learn. Represent.*, 2021.
- [55] Y. Liu et al., "ITransformer: Inverted transformers are effective for time series forecasting," in *Proc. 12th Int. Conf. Learn. Represent.*, 2023.
- [56] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *Int. J. Forecasting*, vol. 36, no. 3, pp. 1181–1191, Jul. 2020.
- [57] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.